

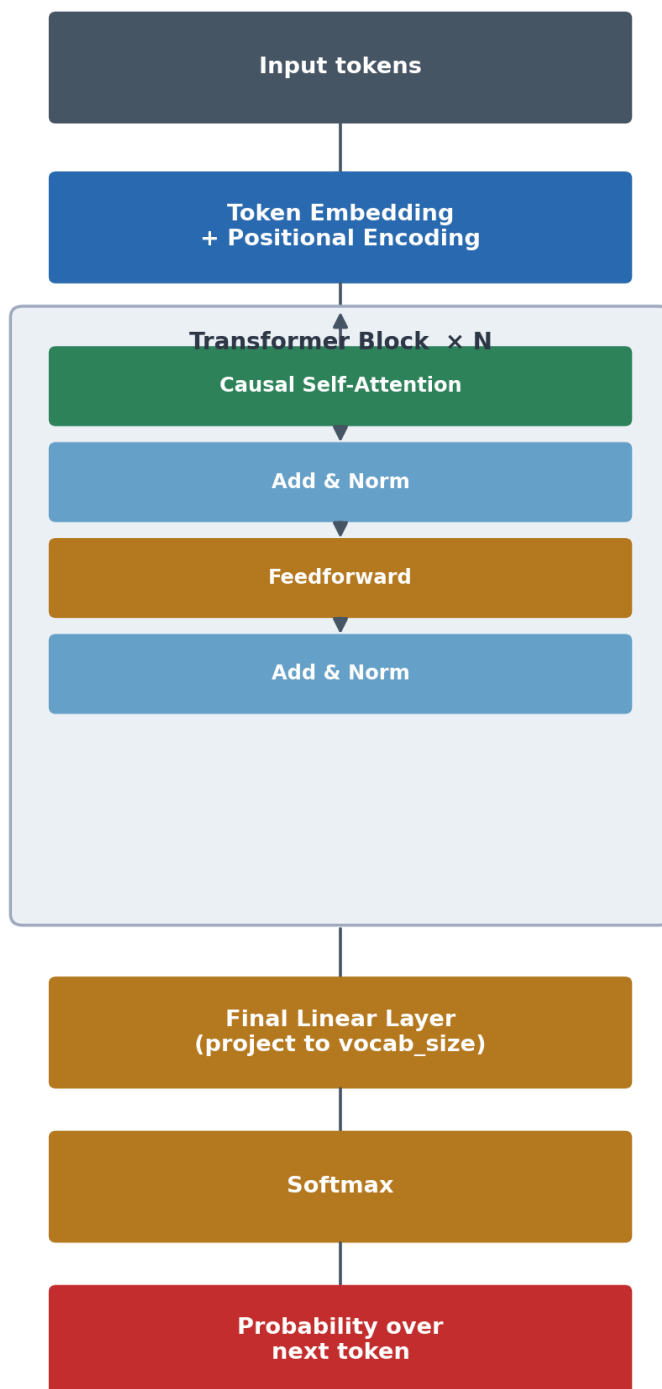
GPT Crash Course

How a decoder-only Transformer becomes a text generator

1. What GPT Actually Is

GPT (Generative Pretrained Transformer) is nothing architecturally new beyond what you've already built — it's a stack of the same causally-masked Transformer blocks from Stage 8 Part 2, trained on one simple task: given some text, predict the next token. "Generative" describes what you DO with a trained model (generate text one token at a time); "pretrained" describes HOW it was trained (on a large generic corpus before any task-specific use).

The entire architecture, start to finish:



Tokens are embedded, positioned, passed through N stacked Transformer blocks (each with causal self-attention), then projected back to a probability distribution over the vocabulary — the model's guess at what comes next.

2. Why "Decoder-Only"

The original Transformer paper (for translation) used two halves: an encoder (reads the full input, no masking) and a decoder (generates output, causally masked, also attends back to the encoder). GPT keeps only the decoder half — causal masking, no separate encoder — because its one job is generating text left-to-right, not translating between two different sequences.

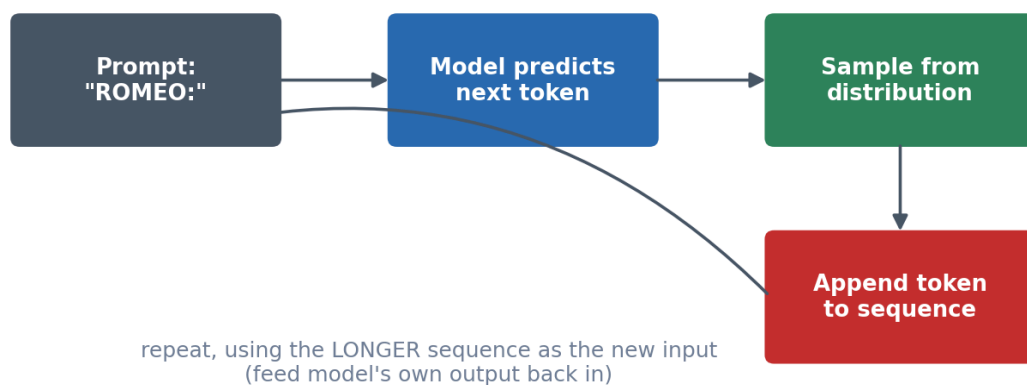
3. Training: Next-Token Prediction

Every training example is just a chunk of real text, paired with that same chunk shifted one position — the target at every position is simply "the next character/token." No labels need to be collected; the training signal is the text itself. This is why GPT-style pretraining can use enormous amounts of raw text with no manual annotation — a huge practical advantage over supervised learning, which needs labels for every example.

Loss: standard cross-entropy between predicted next-token distribution and the actual next token, at every position in the sequence simultaneously.

4. Generation: Autoregressive Sampling

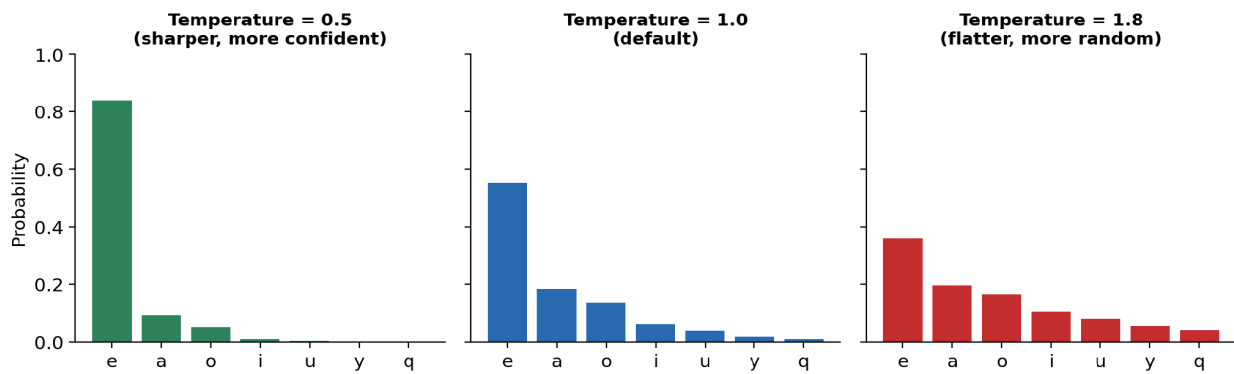
Once trained, generating text means repeatedly: predict a probability distribution over the next token, pick one, append it, and feed the longer sequence back in.



This loop is exactly what causal masking was built for — a model trained to only ever look backward is naturally suited to producing output one step at a time.

5. Sampling Strategies

- **Greedy** — Always pick the highest-probability token. Deterministic, often repetitive.
- **Temperature** — Divide logits by T before softmax. $T < 1$ sharpens the distribution (safer, more repetitive); $T > 1$ flattens it (more diverse, more mistakes).
- **Top-k** — Only sample from the k most likely tokens, discarding the long tail entirely.



Same underlying logits, three temperatures. Low temperature concentrates probability on the top choice; high temperature spreads it out.

6. Scaling: What Changes, What Doesn't

Going from a small character-level model to a production LLM changes scale, not the core mechanism:

- **More layers** — More Transformer blocks stacked (dozens to over a hundred, vs. 4 in your capstone).
- **Bigger `d_model`** — Larger embedding/hidden dimensions (thousands, vs. 128 in your capstone).
- **Better tokenization** — Subword/BPE instead of character-level (Part 1) — shorter sequences per unit of text, larger but more efficient vocabulary.
- **Vastly more training data** — Billions to trillions of tokens, vs. ~1MB here.

What stays identical: embeddings, positional encoding, Q/K/V self-attention, causal masking, residual connections, layer norm, next-token training objective, autoregressive sampling. Every piece in Section 1's diagram is the same piece inside GPT-4 — just repeated far more times, at far larger scale.

7. Quick Glossary

Term	Meaning
GPT	Generative Pretrained Transformer — a decoder-only Transformer trained to predict the next token, then used to generate text one token at a time.
Decoder-only	Uses only causally-masked Transformer blocks (no separate encoder) — every token can only see itself and earlier tokens.
Autoregressive generation	Producing output one token at a time, feeding each new token back in as input for predicting the next.
Logits	A model's raw output scores before softmax — one per vocabulary token.
Temperature	A scaling factor applied to logits before softmax; low = sharper/more confident output, high = flatter/more random.
Top-k sampling	Restricts sampling to only the k most likely next tokens, discarding the rest before sampling.
Context window	The maximum sequence length a model can process at once (<code>block_size</code> in your notebook).
Pretraining	Training on a large, generic text corpus with next-token prediction, before any task-specific fine-tuning.

8. Where This Connects to Your Roadmap

Your Part 3 capstone builds exactly the pipeline in Section 1's diagram — reusing your Part 2 `TransformerBlock` unchanged, stacked N times, trained with the next-token objective from Section 3, and sampled from using the temperature/top-k strategies in Section 5. The honest-evaluation section of that notebook exists because of Section 6 here: your model is real, but small — the gap between it and a production LLM is scale, not a missing architectural idea.